Probeklausur Informatik I
Wintersemester 2006/2007
Prof. Dr. Carsten Damm
08 und 12 Februar 2006
Bearbeitungszeit: 90 Minuten

Vorname:

Nachname:

Matrikelnummer:

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie auf **jedes Blatt** oben Ihren Namen und Ihre Matrikelnummer. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.; Bleistift ist nicht erlaubt.

Zum **Bestehen** der Klausur sind **45** Punkte hinreichend.

☐ meine Note soll mit Matrikelnummer so bald wie möglich auf der Vorlesungs-Webseite veröffentlicht werden.

☐ meine Note soll nicht veröffentlicht werden; ich erfahre sie dann aus Munopag/Wopag (bzw. beim Prüfungsamt der Informatik).

	Max. P.	Schätzung
Aufgabe 1 (Binäre Zahlendarstellung)	11	8
Aufgabe 2 (Grammatik)	14	9
Aufgabe 3 (Divide and Conquer / Aufwand und Komplexität)	20	13
Aufgabe 4 (Java: eine einfache Klasse)	14	7
Aufgabe 5 (Sortierverfahren, Schleifeninvariante und Java)	21	14
Aufgabe 6 (Fibonacci-Funktion und Algorithmen-Entwurf)	10	6
Summe	90	57

Note:

Aufgabe 1 (Binäre Zahlendarstellung [11 Punkte])
Gegeben sind die Zahlen „122“ und „89“

a) Stellen Sie beide als Binärzahlen sowie als Hexadezimalen dar. (3 P)

Lösung

122:	01111010	= Hex 7A
89:	01011001	= Hex 59

b) Addieren Sie beide Zahlen im Binärformat und verifieren Sie das Ergebnis indem Sie es in das Dezimalformat zurückwandeln. (4 P)

Lösung

122:	01111010		
89:	01011001		
Add: 211:	11010011	$= 128 + 64 + 16 + 2 + 1$	

c) Stellen Sie „-89“ als Zweierkomplement dar und berechnen Sie „$122 - 89$“. (4 P)

Lösung

Zweierkomplement von 89: 89 ist		01011001	
	Einerkomplement:	10100110	
	$+1$:	10100111	ist das Zweierkomplement
Addieren:	01111010		
$+$	10100111		
Summe:	00100001	$= 122 - 89 = 33$	

Aufgabe 2 (Grammatik [14 Punkte])

a) Geben Sie eine Grammatik in EBNF (*erweiterte Backus-Naur-Form*) für die Menge aller Palindrome aus dem Alphabet $\{a, b\}$ an (Palindrome sind Ausdrücke, die sowohl vorwärts als auch rückwärts dasselbe Wort ergeben, z.B. *abba*, *bbabb*). (5 P)

Lösung

Die Grammatik wird definiert über:

$G = \{N, T, P, S\}$ oder $G = \{V, \Sigma, P, S\}$
wobei
V = Variablenmenge (N = Nichtterminalsymbol-Menge) = $\{S\}$
Σ = Terminalalphabet (T = Terminalsymbol-Menge) = $\{\epsilon, a, b\}$
P = Produktionsregelnmenge
S = Startsymbol

Für P gilt:

$S := a|b|aSa|bSb|\epsilon$ (EBNF-Darstellung)

b) Leiten Sie mit den Produktionsregeln Ihrer Grammatik das Wort *aababaa* aus dem Startsymbol ab. (4 P)

Lösung

$S \rightarrow aSa \rightarrow\rightarrow aaSaa \rightarrow aabSbaa \rightarrow aababaa$

c) Geben Sie einen deterministischen endlichen Automaten an der alle Palindrome aus dem Alphabet $\{a, b\}$ mit einer Länge von mindestens 1 bis maximal 4 Zeichen erkennt. (5 P)

Lösung

Siehe Automat auf Beiblatt 1

Aufgabe 3 (Divide and Conquer / Aufwand und Komplexität [20 Punkte])
Gegeben sei ein Divide-and-Conquer-Algorithmus, der auf eine Folge von n Zahlen wie folgt angewendet wird:

- falls $n = 1$, gib die Zahl zurück
- sonst:
  1. Löse dasselbe Problem für die aus den ersten 1 ... $\frac{n}{2}$ Zahlen bestehende Teilfolge, erhalte einen Wert x_1,
  2. Löse dasselbe Problem für die aus den restlichen $(\frac{n}{2} + 1)$ ... n Zahlen bestehende Teilfolge, erhalte einen Wert x_2,
  3. Löse dasselbe Problem für die aus den mittleren $(\frac{n}{4} + 1)$ ... $\frac{3}{4}n$ Zahlen bestehende Teilfolge, erhalte einen Wert x_3,
  4. Löse dasselbe Problem für die aus den $(\frac{3}{4}n + 1)$ ...n sowie 1 ... $\frac{1}{4}n$ Zahlen bestehende Teilfolge, erhalte einen Wert x_4,
  5. mache eine Nachverarbeitung über alle n Zahlen, die n^2 Schritte benötigt, erhalte einen Wert y,
  6. Gib $max(x_1, x_2, x_3, x_4, y)$ zurück.

Tipp: Schätzen Sie die Kostenfunktion sinnvoll nach oben hin ab, um Theoreme aus der Vorlesung benutzen zu können!

a) Geben Sie die Rekurrenzgleichung für $T(n)$ an. (6 P)

Lösung

$T(n) = 4T(\frac{n}{2}) + n^2 + 5$ für $n > 1$
$T(1) = 1$

Anmerkung: einige Lösungen zielten auf $T(n) = 3T(\frac{n}{2}) + 2T(\frac{n}{4}) + n^2 + 5$.
In Punkt 4) wird explizit gesagt, dass es sich um *eine* Teilfolge handelt, die aus den beiden Fragmenten zusammengesetzt ist.

b) Welche asymptotische Laufzeit hat der Algorithmus (mit Begründung). (5 P)

Lösung

Die Konstante „+5“ kann man direkt in $T(n) = 4T(\frac{n}{2}) + O(n^2)$ mit einbeziehen.
Master-Theorem mit $a = 4$, $b = 2$ und $k = 2$, also $a = b^k$ und damit Lösung $O(n^k \log n)$, also in diesem Fall $O(n^2 \log n)$.

c) Jemand optimiert den Algorithmus nun, und stellt fest, dass Schritt (4) weggelassen werden kann, und Schritt (5) in $O(n \log n)$ erledigt werden kann. Geben Sie wieder die Rekurrenzgleichung an. (4 P)

Lösung

$T_2(n) = 3T_2(\frac{n}{2}) + O(n \log n) + 4$ für $n > 1$
$T_2(1) = 1$

d) Geben Sie eine obere Schranke für die asymptotische Laufzeit des optimierten Algorithmus an? (5 P)

Lösung

Wieder fällt die Konstante „+4“ in das $O(n \log n)$

Zuerst einmal: da es eine Optimierung des ersten Algorithmus ist, gilt auf jeden Fall $T_2(n) = O(n^2 \log n)$.

Master-Theorem mit $a = 3$, $b = 2$, k kann als beliebige Konstante $k > 1$ angenommen werden ($\log n = O(n^k)$ für alle $k > 1$). Also $a > b^k$ und damit $T(n) = O(n^{log_b(a)}) = O(n^{log_2(3)}) \approx O(n^{1.58})$.

Aufgabe 4 (Java: eine einfache Klasse [14 Punkte])
Geben Sie eine einfache Klasse `Person` an, mit der Personen abgespeichert werden können. Jede Person zeichnet sich durch ihren Namen aus, sowie ihr Gewicht. Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und des Gewichtes,
- Setzen und Abfragen des Gewichtes jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele diese zusammen wiegen.

Lösung

```
public class Person {
  String name;
  long gewicht;      // oder int oder sonstwas
  static int anzahl;
  static long gesamtgewicht;

  public Person(String t){
    name = t;
    anzahl++;
  }

  public Person(String t, long p) {
    name = t;
    anzahl++;
    gewicht = p;
    gesamtgewicht = gesamtgewicht + p;
  }

  public void setGewicht(long p) {
    gesamtgewicht = gesamtgewicht - gewicht + p;
    gewicht = p;
  }

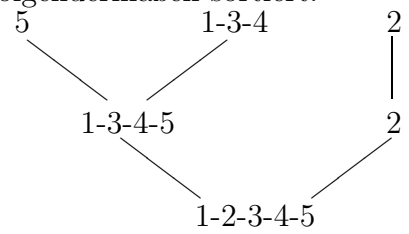
  public void getGewicht(long p) {
    return gewicht;
  }

  public static int wieviele() {
    return anzahl;
  }
```

```
  public static long gesamtgewicht() {
    return gesamtgewicht;
  }
}
```

Aufgabe 5 (Sortierverfahren, Schleifeninvariante und Java [21 Punkte])

a) Betrachten Sie folgende Variante von MergeSort:
Anstatt eine Zahlenfolge auf ihre einzelnen Elemente herunter zu brechen und diese dann mittels Reißverschlussverfahren zusammenzufügen, werden bereits sortierte Teilfolgen gesucht und dann vereint. Z. B. Wird 5,1,3,4,2 folgendermaßen sortiert:



Angenommen, eine Funktion `static public void merge (int[] a, int l, int m, int r)` , die auf dem Feld `a[]` die sortierten Bereiche von l bis m und $m+1$ bis r zusammenfügt, sei bereits implementiert. Implementieren Sie diese MergeSort-Variante in Java. (6 P)

(Hinweis: Es ist weder eine Main-Funktion, noch eine Klasse notwendig zu schreiben, beschränken Sie sich lediglich auf die Funktion selbst. Die Funktion muss nicht effektiv sein, sondern sollte möglichst einfach geschrieben werden.)

Lösung

```
public static void sort (int[] a){
  int l,m,r;

  //1. Schritt: linke Folge suchen
  l=0;
  m=l+1;
  while (m!=a.length&&a[m-1]<=a[m]) m++;

  if (m!=a.length){
    //2. Schritt: rechte Folge bestimmen
    r=m+1;
    while (r!=a.length&&a[r-1]<=a[r]) r++;

    //3. Schritt: merge mit korrekten Parametern
    merge(a,l,m-1,r-1);
    sort(a);
  }
}
```

b) Gegeben ist das folgende Programm:

```
public class Mystery {
  public static int compute(int n) {
```

```
    int i = 1;
    int sq = 1;

    while (i<n)
    {
      sq = sq + 2*i + 1;
      i = i + 1;
    }
    return sq;
  }
}
```

1. Was berechnet der Aufruf `Mystery.compute(n)` ? (3 P)
   Hinweis: probieren Sie es für n = 1, ... , 5 aus

   Lösung

   Mystery berechnet n^2 : $1^2 = 1$, $(i+1)^2 = i^2 + 2i + 1$ (Binomische Formel)

2. Geben Sie für die Schleife die Vorbedingung, Schleifeninvariante und Nachbedingung an - beweisen Sie damit induktiv Ihre Aussage in Aufgabenteil 1. (6 P)

   Lösung

   - Vorbedingung und Schleifeninvariante: $sq = i^2$
   - Schleifenbedingung: $i < n$
   - Nachbedingung: $sq = n^2$
   - Beweis, dass Vorbedingung von der Initialisierung garantiert wird:
     $1 = 1^2$
   - Beweis der Schleifeninvariante (induktiv):
     Am Anfang des Schleifenkörpers gilt (Aussage für i): $sq = i^2$.
     Dann wird sq um $2i + 1$ erhöht, und dann wird noch i um 1 erhöht.
     Damit gilt jetzt wieder $sq = i^2 + 2i + 1 = (i+1)^2$
   - Beweis der Nachbedingung (= Schleifeninvariante + negierte Schleifenbedingung):
     $sq = i^2$ und $i = n$, daraus folgt $sq = n^2$

c) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1P. (6 P)

Für jede falsche Antwort wird ein Punkt abgezogen! Jede nicht gegebene Antwort wird aber nur als 0P gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0P gewertet.

Geben Sie keine zusätzlichen Erläuterungen oder Einschränkungen zu Ihrer Auswahl an!

Aussage	**Richtig**	**Falsch**
Java		
Eine Klasse, die keine Methoden besitzt, ist ein Interface.		
Eine Klasse darf keine 2 Methoden mit gleicher Signatur enthalten.		
`class A extends B implements C,D {...}` ist korrekter Java-Code		
Sortieren und Suchen		
Mit n Vergleichen kann man prüfen, ob n+1 Zahlen geordnet sind.		
Binärsuche ist ein Divide & Conquer-Verfahren.		
Eine Queue arbeitet nach dem LILO-Prinzip. (LILO = *Last In Last Out*)		

Lösung

Aussage	**Richtig**	**Falsch**
Java		
Eine Klasse, die keine Methoden besitzt, ist ein Interface.		*X*
Eine Klasse darf keine 2 Methoden mit gleicher Signatur enthalten.		*X*
`class A extends B implements C,D {...}` ist korrekter Java-Code	*X*	
Sortieren und Suchen		
Mit n Vergleichen kann man prüfen, ob n+1 Zahlen geordnet sind.	*X*	
Binärsuche ist ein Divide & Conquer-Verfahren.	*X*	
Eine Queue arbeitet nach dem LILO-Prinzip. (LILO = *Last In Last Out*)	*X*	

Aufgabe 6 (Fibonacci-Funktion und Algorithmen-Entwurf [10 Punkte])
Gegeben sei die Fibonacci-Funktion mit

- $fib(0) := fib(1) := 1$
- $fib(n+1) := fib(n) + fib(n-1)\ \forall n \in \mathrm{N},\ n > 1$

a) Berechnen Sie $fib(5)$. (2 P)

Lösung

$fib(5) = fib(4) + fib(3) = fib(3) + fib(2) + fib(2) + fib(1) = fib(2) + fib(1) + fib(1) + fib(0) + fib(1) + fib(0) + fib(1) = fib(1) + fib(0) + fib(1) + fib(1) + fib(0) + fib(1) + fib(0) + fib(1) = 8$

b) Wie lautet der größte gemeinsame Teiler (ggT) von $fib(4)$ und $fib(5)$? (3 P)

Lösung

$fib(5) = 8, fib(4) = 5 \Rightarrow ggT(5,4) = 1 \Rightarrow fib(4)$ und $fib(5)$ sind teilerfremd.

Gegeben ist eine nur aus „+“ und „−“ bestehende endliche Zeichenfolge, z.B. „− − + + + − − + + + + + − + − + − + − + − − +“
(Sie können annehmen, dass die Zeichenfolge als Array z[0]...z[k] gegeben ist).

c) Geben Sie einen Algorithmus an, der ausgibt, wie lang die längste Teilsequenz ist, die nur aus „+“-Zeichen besteht (im Beispiel 5). (5 P)
Sie können dies entweder in Java, als Flussdiagramm, oder als Pseudocode tun.

Lösung

Initialisierung: $i = 0,\ n = 0,\ max = 0$
Schritt für Schritt nach rechts durchlaufen.

- Falls die Zahl an Stelle i ein „+“ ist, $n = n + 1$,
- Falls die Zahl an Stelle i ein „−“ ist,
  falls $n > max$, setze $max = n$
  setze $n = 0$.